

UNIT-1

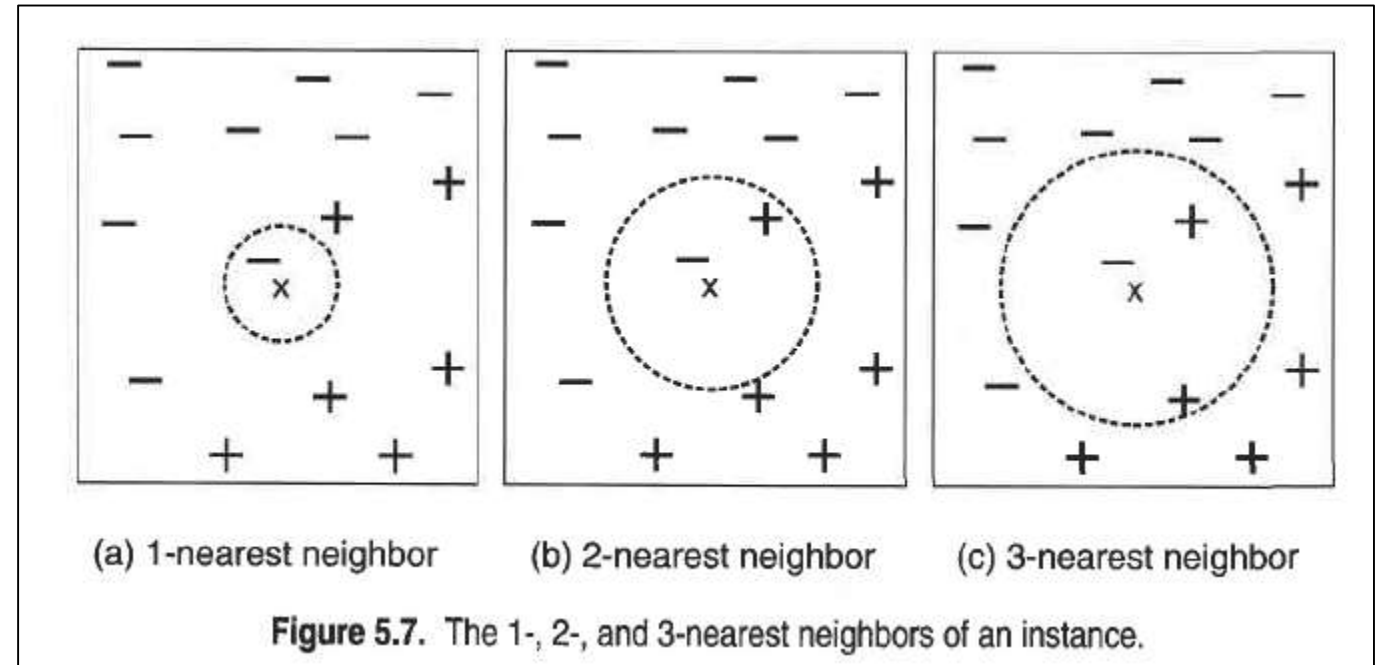
K-NEAREST NEIGHBOR

K-NN Topics

- Introduction
- Problems on k-NN
- Algorithm
- Characteristics of k-NN

Introduction

- **Eager learners like Decision Tree and Rule Based Classifier** learn a model that **maps input attributes to class label** as soon as **training data becomes available**.
- Lazy Learners like k-NN delay the process of modelling the training data until it is needed to classify the test records.
- K-NN classifies a record (data point) by considering classes of nearest neighbours. It compares the differences.
- Figure 5.7 illustrates the 1-, 2-, and 3-nearest neighbors of a data point located at the center of each circle. The data point is classified based on the class labels of its neighbors. In the case where the neighbors have more than one label, the data point is assigned to the majority class of its nearest neighbors.



- In Figure 5.7(a), the 1-nearest neighbor of the data point is a negative example. Therefore, the data point is assigned to the negative class. If the number of nearest neighbors is three, as shown in Figure 5.7(c), then the neighborhood contains two positive examples and one negative example.
- Using the majority voting scheme, the data point is assigned to the positive class. In the case where there is a tie between the classes (see Figure 5.7(b)), we may randomly choose one of them to classify the data point.

k-NN Problem

1. Using k- nearest neighbour algorithm, identify the class for

Test Data (T) : <Acid Durability=3, Strength=7, Class=?> using following training data set for k=1 and k=2.

Name	Acid Durability	Strength	Class	
Type 1	7	7	Bad	→ T1
Type 2	7	4	Bad	→ T2
Type 3	3	4	Good	→ T3
Type 4	1	4	Good	→ T4

Ans:

- Using Euclidean Distance: $d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$
- Distance d(Test-Data (T), Training-Data (T1,T2,T3,T4) using Euclidean Distance
 - ✓ $d(T, T1) = \sqrt{(3 - 7)^2 + (7 - 7)^2} = \sqrt{16} = 4 \rightarrow$ Rank 3
 - ✓ $d(T, T2) = \sqrt{(3 - 7)^2 + (7 - 4)^2} = \sqrt{25} = 5 \rightarrow$ Rank 4
 - ✓ $d(T, T3) = \sqrt{(3 - 3)^2 + (7 - 4)^2} = \sqrt{9} = 3 \rightarrow$ Rank 1
 - ✓ $d(T, T4) = \sqrt{(3 - 1)^2 + (7 - 4)^2} = \sqrt{13} = 3.6 \rightarrow$ Rank 2
- When k=1, (nearest 1-neighbor T3) since class of T3=Good, Test Data belongs to class = Good
- When k=2, (nearest 2-neighbors are T3 and T4) since class of T3 and T4=Good, Test Data belongs to class = Good

k-NN Problem

2. Using k- nearest neighbour algorithm, identify the class for

Test Data (T) : <Acid Durability=4, Strength=8, Class=?> using following training data set for k=1 and k=2.

Name	Acid Durability	Strength	Class	
Type 1	4	3	Good	→ T1
Type 2	6	5	Bad	→ T2
Type 3	3	7	Bad	→ T3
Type 4	2	8	Good	→ T4

Ans:

- Using **Euclidean Distance**: $d(p, q) = \sqrt{\sum_{i=1}^n (p_i - q_i)^2}$
- Distance d(Test-Data (T), Training-Data (T1,T2,T3,T4) using Euclidean Distance
 - ✓ $d(T, T1) = \sqrt{(4 - 4)^2 + (8 - 3)^2} = \sqrt{25} = 5 \rightarrow$ Rank 4
 - ✓ $d(T, T2) = \sqrt{(4 - 6)^2 + (8 - 5)^2} = \sqrt{13} = 3.6 \rightarrow$ Rank 1
 - ✓ $d(T, T3) = \sqrt{(4 - 3)^2 + (8 - 7)^2} = \sqrt{2} = 1.41 \rightarrow$ Rank 3
 - ✓ $d(T, T4) = \sqrt{(4 - 2)^2 + (8 - 8)^2} = \sqrt{4} = 2 \rightarrow$ Rank 2
- When k=1, (nearest 1-neighbor T3) since class of T3=Bad, Test Data belongs to class = Bad
- When k=2, (nearest 2-neighbors are T3 and T4) since class of T3=Bad and T4=Good, Test Data can be assigned either Good or Bad class.

K-Nearest Neighbor: Selecting 'k' value

- If **k** is too small → K-NN is susceptible to **overfitting** because of **noise in the training data**.
- If **k** is too large, the nearest-neighbor classifier may **misclassify** the **test instance** because its list of nearest neighbors may include data points that are located far away from its neighborhood (Fig below).

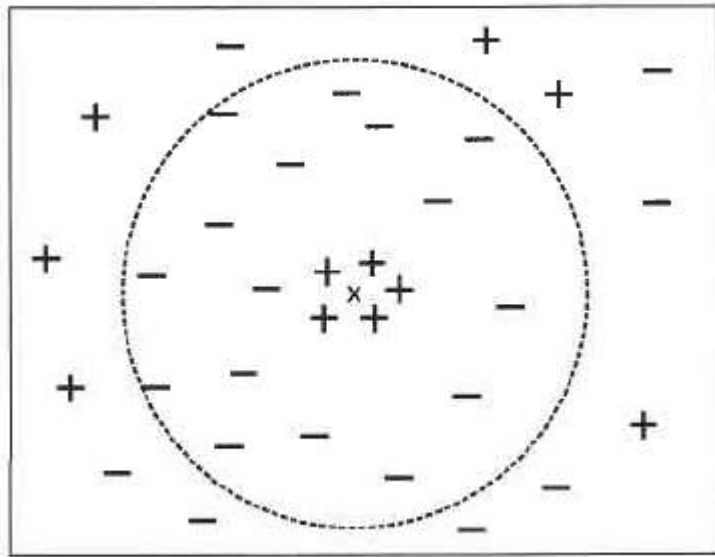


Figure 5.8. k -nearest neighbor classification with large k .

k-NN algorithm

- The algorithm computes the distance (or similarity) between each test example, $z = (x', y')$ and all the training examples $(x, y) \in D$ to determine its nearest-neighbor list, D_z .
- Once the nearest-neighbor list is obtained, the test example is classified based on the majority class of its nearest neighbors.
- In the majority voting approach, every neighbor has the same impact on the classification. This makes the algorithm sensitive to the choice of k .
- So, **distance weighted voting** is used where weight ' w_i ' is assigned to each neighbor based on distance $w_i = \frac{1}{d(x', x_i)^2}$
- So, training records located far away from ' z ' have a weaker impact on classification.

Algorithm 5.2 The k -nearest neighbor classification algorithm.

- 1: Let k be the number of nearest neighbors and D be the set of training examples.
 - 2: for each test example $z = (x', y')$ do
 - 3: Compute $d(x', x)$, the distance between z and every example, $(x, y) \in D$.
 - 4: Select $D_z \subseteq D$, the set of k closest training examples to z .
 - 5: $y' = \operatorname{argmax}_v \sum_{(x_i, y_i) \in D_z} I(v = y_i)$
 - 6: end for
-

Majority Voting: $y' = \operatorname{argmax}_v \sum_{(x_i, y_i) \in D_z} I(v = y_i),$

Where:

- ✓ $v \rightarrow$ class label,
- ✓ $y_i \rightarrow$ class label for one of the nearest neighbors,
- ✓ $I(.) \rightarrow$ indicator function that returns the value 1 if its argument is true and 0 otherwise.

5.2.2 Characteristics of Nearest-Neighbor Classifiers

1. Nearest-neighbor classification is part of a more general technique known as **instance-based learning**, which uses **specific training instances** to make predictions **without** having to maintain an abstraction (or **model**) derived from data. Instance-based learning algorithms require a **proximity measure** to **determine the similarity or distance between instances** and a **classification function** that returns the predicted class of a test instance based on its proximity to other instances.
2. **Lazy learners** such as nearest-neighbor classifiers **do not require model building**. However, **classifying a test example** can be quite **expensive** because we need to **compute the proximity values individually** between **the test and training examples**. In contrast, **eager learners** often **spend** the bulk of their **computing resources for model building**. Once a **model** has been **built**, **classifying a test example is extremely fast** (E.g.: **Decision Tree**).

5.2.2 Characteristics of Nearest-Neighbor Classifiers

3. Nearest-neighbor classifiers make their **predictions** based on **local information**, whereas **decision tree and rule-based classifiers** attempt to find a **global model** that **fits the entire input space**. Because the classification decisions are made locally, **nearest-neighbor classifiers (with small values of k)** are quite **susceptible to noise**.
4. Nearest-neighbor classifiers can produce **arbitrarily shaped decision boundaries**. Such boundaries provide a **more flexible model representation compared to decision tree and rule-based classifiers** that are often constrained to **rectilinear decision boundaries**. The decision boundaries of nearest-neighbor classifiers also have **high variability** because they **depend on the composition of training examples**. **Increasing the number of nearest neighbors may reduce such variability**.

5.2.2 Characteristics of Nearest-Neighbor Classifiers

5. Nearest-neighbor classifiers can **produce wrong predictions unless the appropriate proximity measure and data preprocessing steps are taken**. For example, suppose we want to classify a group of people based on **attributes such as height (measured in meters) and weight (measured in pounds)**. The **height attribute has a low variability**, ranging from 1.5 m to 1.85 m, whereas the **weight attribute may vary** from 90 lb. to 250 lb. If the scale of the attributes are not taken into consideration, the **proximity measure may be dominated by differences in the weights of a person**.